Universidad de La Habana
Facultad de Matemática y Computación

Ingeniería de Software

Seminario de Arquitecturas de Software:

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Equipo: Base de Batos

Fecha de entrega: 23 - 10 - 2026

Integrante	**Pregunta Asignada**
Gabriel Pérez Suárez	1
Alejandro Lopez Castro	2
Héctor Duarte Vázquez	3
Bryan Ernesto Moya Aquino	4
Alex Leonardo Cuervo Grillo	5
Fabio Hernández Piloto	6

Repositorio público del proyecto:
`https://github.com/el-M0ya/Base_de_Batos`

Índice

Desarrollo de las preguntas **1**

1. Análisis de Trade-offs (Arquitectura N-Capas vs. Clean/Onion/Hexagonal) **1**
1.1. ¿Cuál es el error fundamental de acoplamiento? . . . 1
1.2. ¿Cómo resolvería el problema la Arquitectura Limpia? . . . 2
1.3. ¿Cómo resolvería el problema la Arquitectura Hexagonal? . . . 3
1.4. Impacto en la testabilidad . . . 4
1.5. ¿Cómo cambiar de SQL Server a MongoDB sin afectar la lógica de negocio? . . . 5
1.6. Conclusión . . . 6

2. Comparación de Estilos: La Simetría del Hexágono vs. la Jerarquía de las Capas **7**
2.1. Análisis conceptual: simetría hexagonal frente a capas . . . 7
2.2. Traducción de «Puerto y Adaptador» en la práctica de definir interfaces en las capas internas de la Arquitectura Onion . . . 7
2.3. El verdadero propósito del hexágono: Eliminar la asimetría izquierda-derecha o arriba-abajo . . . 9

3. Aplicación de DDD y Bounded Context en un Dominio Complejo **10**
3.1. Descomposición del sistema en contextos delimitados . . . 10
3.2. Prevención de la contaminación del modelo de dominio . . . 11
3.3. Comunicación desacoplada entre contextos . . . 11
3.4. Alineación con arquitecturas orientadas al dominio y adaptadores de servicio . . . 12

4. Evaluación de la Inversión de Dependencias en la Práctica **14**
4.1. a) ¿Por qué la dependencia directa viola los principios de estas arquitecturas? . . . 15
4.1.1. Análisis del diseño actual . . . 15
4.1.2. Violación del Principio de Inversión de Dependencias (DIP) . . . 15
4.1.3. Violación de la Regla de Dependencia . . . 15
4.1.4. Relación con las arquitecturas Hexagonal, Limpia y Onion . . . 16
4.2. b) Rediseño de la interacción entre `OrderProcessor` y `AddressValidatorService` 16
4.2.1. Definición de la solución . . . 16
4.2.2. Diagrama de la arquitectura propuesta . . . 16
4.2.3. Ejemplo de implementación . . . 18
4.3. c) Efectos sobre las pruebas, la mantenibilidad y el cambio de proveedor . . . 19
4.3.1. Pruebas unitarias aisladas . . . 19
4.3.2. Mantenibilidad y evolución tecnológica . . . 19
4.3.3. Implicaciones para el futuro . . . 19

5. Análisis de un Problema Real: el "Kitty Problem" y el Principio de Abierto/Cerrado **21**
5.1. Análisis de la Causa Raíz . . . 22

5.2. Rediseño basado en DDD y Clean Architecture 22
5.2.1. Abstracción del Núcleo 22
5.2.2. Componentes Verticales por Dominio 22
5.2.3. Patrones de Diseño 23
5.3. Cumplimiento del Principio de Abierto/Cerrado (OCP) 23
5.4. Rol de los Adaptadores y Fábricas 24

6. Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad 25
6.1. Capas abiertas y degradación del Aislamiento de Capas 25
6.2. El Antipatrón del Sumidero (*Sinkhole Anti-pattern*) y la Regla 80/20 26
6.3. Comparación con las arquitecturas Limpia y Hexagonal 27

Referencias Bibliográficas 29

Declaración de uso de inteligencia artificial 30

Desarrollo de las preguntas

1. Análisis de Trade-offs (Arquitectura N-Capas vs. Clean/Onion/Hexagonal)

Un equipo de desarrollo está construyendo un sistema de gestión de inventario. Inicialmente, eligen una Arquitectura N-Capas tradicional por su simplicidad y familiaridad. Después de un año, el sistema se ha vuelto un "monolito" difícil de mantener, donde los cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario, y las pruebas unitarias son lentas y frágiles.

Con base en los principios de la Arquitectura Limpia y la Arquitectura Hexagonal, explique detalladamente qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adaptadores (Cockburn) resolverían estructuralmente este problema. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

Respuesta

1.1. ¿Cuál es el error fundamental de acoplamiento?

El problema principal del sistema de gestión de inventario es que existe un **acoplamiento excesivo entre la lógica de negocio y los detalles de infraestructura**, como la base de datos y la interfaz de usuario.

En una arquitectura tradicional de N-Capas, el sistema suele organizarse en tres capas principales:

- **Capa de presentación:** contiene las interfaces con las que interactúa el usuario.
- **Capa de negocio:** implementa las reglas y operaciones del sistema, como registrar productos, actualizar existencias y comprobar la disponibilidad de artículos.
- **Capa de datos:** se encarga de almacenar y recuperar la información de la base de datos.

En el diseño tradicional, las dependencias suelen dirigirse desde las capas superiores hacia las inferiores. Esto puede provocar que la lógica de negocio dependa directamente de tecnologías concretas de persistencia o de mecanismos específicos de presentación.

Por ejemplo, si la lógica de inventario utiliza directamente consultas SQL Server, sus cambios pueden repercutir en las operaciones de negocio. De igual manera, si las reglas de negocio están mezcladas con el código de la interfaz de usuario, modificar esta última puede obligar a modificar componentes que deberían permanecer independientes.

Este diseño genera varios problemas:

1. **Baja mantenibilidad:** un cambio en una tecnología puede obligar a modificar múltiples componentes del sistema.

2. **Dificultad para realizar pruebas unitarias:** las pruebas de la lógica de negocio pueden requerir una base de datos real o una configuración compleja del entorno.
3. **Rigidez arquitectónica:** sustituir una tecnología o incorporar una nueva funcionalidad puede provocar cambios en partes del sistema que no deberían verse afectadas.
4. **Propagación de cambios:** los componentes están tan relacionados que una modificación aparentemente pequeña puede generar errores en otras partes de la aplicación.

Por tanto, el error no consiste simplemente en utilizar una arquitectura de N-Capas, sino en organizar las dependencias de forma que **el núcleo del negocio quede condicionado por los detalles técnicos externos**.

La figura 1 ilustra una implementación en la que los detalles de persistencia se filtran hacia la lógica de negocio y la presentación. Esta situación representa un problema de diseño, no una consecuencia inevitable de utilizar N-Capas.

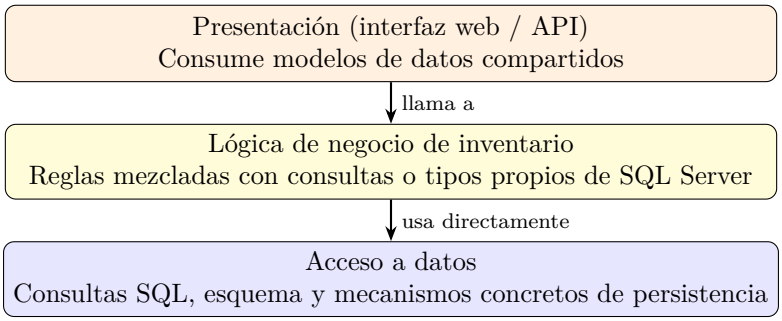


Figura 1: Ejemplo de N-Capas con fuga de detalles de persistencia hacia la lógica de negocio y la presentación. El problema descrito es una implementación acoplada, no una consecuencia inevitable de toda arquitectura por capas.

1.2. ¿Cómo resolvería el problema la Arquitectura Limpia?

La Arquitectura Limpia, propuesta por Robert C. Martin, establece la denominada **Regla de Dependencia**: las dependencias del código fuente deben apuntar hacia el interior, hacia las políticas y reglas de negocio de mayor nivel.

Esto significa que las capas externas, como la interfaz de usuario, la base de datos y los servicios externos, pueden depender del núcleo del sistema, pero el núcleo no debe depender directamente de ellas.

Para aplicarlo al sistema de inventario, se podrían distinguir los siguientes componentes:

- **Entidades del dominio:** representan conceptos fundamentales, como Producto, Existencia e Inventario, junto con las reglas de negocio que les correspondan.
- **Casos de uso:** implementan operaciones concretas, como registrar un producto, consultar existencias o actualizar el inventario.
- **Adaptadores de interfaz:** transforman los datos procedentes de la interfaz de usuario o los preparan para su presentación.

- **Infraestructura:** contiene la implementación de acceso a datos, los controladores de base de datos y otras tecnologías externas.

La clave consiste en definir abstracciones dentro del núcleo de la aplicación. Por ejemplo, un caso de uso puede necesitar consultar y guardar productos mediante una interfaz `ProductoRepository`, sin conocer cómo se almacenan realmente. La infraestructura implementaría esa interfaz mediante una clase específica, como `SqlServerProductoRepository`.

De esta forma, el caso de uso depende de una abstracción estable, mientras que la implementación concreta de acceso a datos depende de esa misma abstracción.

La figura 2 resume la dirección de las dependencias en Clean Architecture. La Regla de Dependencia establece que el código fuente debe apuntar hacia las políticas internas (Martin 2017).

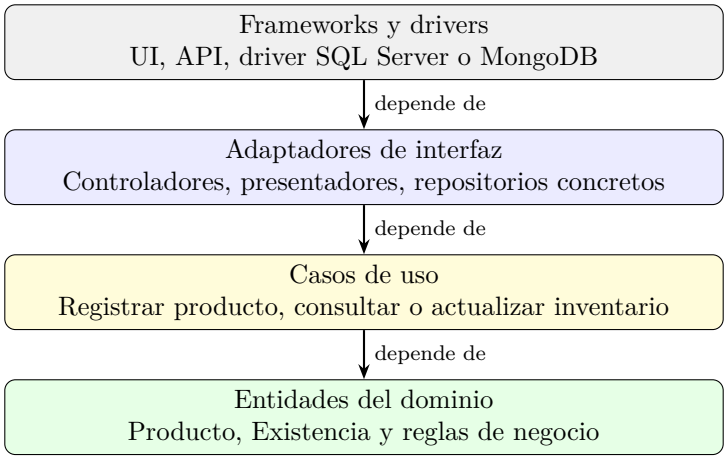


Figura 2: Vista lineal simplificada de Clean Architecture. Las flechas representan dependencias del código fuente hacia las políticas internas, no necesariamente el orden de ejecución.

Esta reorganización permite que las reglas del inventario permanezcan estables incluso cuando cambian las tecnologías utilizadas para almacenar o presentar la información.

1.3. ¿Cómo resolvería el problema la Arquitectura Hexagonal?

La Arquitectura Hexagonal, propuesta por Alistair Cockburn, persigue el mismo objetivo de aislar la lógica de la aplicación, pero utiliza el concepto de **puertos y adaptadores**.

El núcleo de la aplicación contiene las operaciones y reglas del inventario. Los puertos definen las interfaces mediante las cuales el núcleo se comunica con el exterior, mientras que los adaptadores implementan la conexión con tecnologías o sistemas concretos.

En el sistema propuesto, se pueden identificar dos tipos de interacción:

- **Puerto de entrada:** permite solicitar una operación al sistema, por ejemplo, registrar un producto o consultar sus existencias. Una interfaz de usuario o una API puede actuar como adaptador de entrada.
- **Puerto de salida:** permite al núcleo solicitar un servicio externo, como guardar un producto

o recuperar sus datos. Un repositorio definido mediante una interfaz puede representar este puerto, mientras que un adaptador de infraestructura lo implementa para una base de datos específica.

Por ejemplo, el caso de uso `ActualizarInventario` puede utilizar el puerto `ProductoRepository` para guardar los cambios. Un adaptador para SQL Server implementa dicho puerto mediante consultas SQL, mientras que otro adaptador podría utilizar MongoDB.

El caso de uso no necesita conocer ninguno de esos mecanismos concretos. Solo necesita conocer el contrato definido por el puerto.

La figura 3 aplica el concepto de puertos y adaptadores descrito por Cockburn (2005) al sistema de inventario. Para mantener el diagrama legible, se muestran las alternativas de persistencia y pruebas alrededor del puerto de salida.

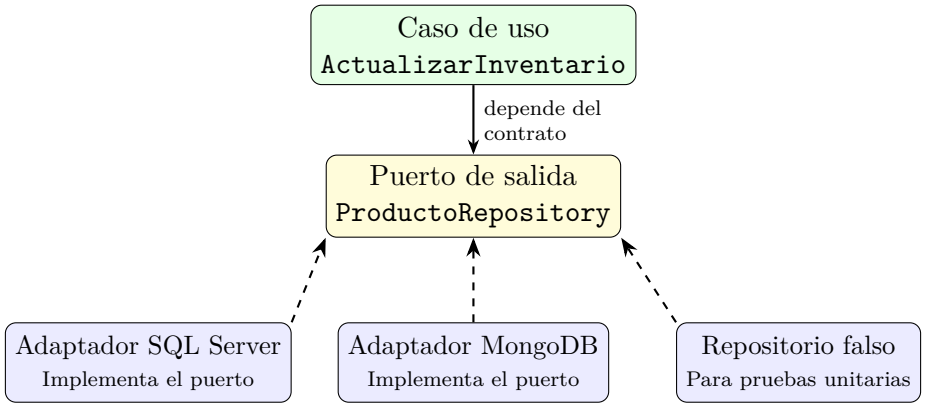


Figura 3: Arquitectura hexagonal aplicada a la persistencia del inventario. El caso de uso depende del puerto definido por la aplicación; los adaptadores de SQL Server, MongoDB o pruebas implementan ese contrato desde fuera.

La diferencia fundamental es que la arquitectura no obliga a que todas las interacciones sigan una jerarquía rígida de capas: **los distintos sistemas externos pueden conectarse al núcleo mediante adaptadores, manteniendo aislada la lógica de negocio**.

1.4. Impacto en la testabilidad

La aplicación de estos principios mejora considerablemente la capacidad de probar el sistema.

En el diseño original, una prueba unitaria de la lógica de inventario podría necesitar conectarse a SQL Server, preparar registros en una base de datos y configurar otros componentes externos. Esto hace que las pruebas sean más lentas, frágiles y difíciles de mantener.

Con Clean o Hexagonal, los casos de uso dependen de interfaces y no de implementaciones concretas. Por ello, durante las pruebas se puede sustituir el repositorio real por una implementación simulada (*mock* o *fake*).

Por ejemplo, para probar la operación de registrar un producto, se puede utilizar un repositorio en memoria que permita comprobar si:

- El producto se registra correctamente.
- Se rechazan datos que incumplen las reglas del negocio.
- Se actualizan las existencias según las condiciones establecidas.
- Se manejan correctamente los errores devueltos por el repositorio.

Estas pruebas no necesitan realizar conexiones reales a la base de datos. Además, los resultados no dependen de la disponibilidad de un servidor externo, lo que facilita su ejecución repetida.

Es importante señalar que las pruebas de integración con la base de datos siguen siendo necesarias. La ventaja es que se pueden realizar por separado, sin obligar a que todas las pruebas de las reglas de negocio dependan de la infraestructura.

1.5. ¿Cómo cambiar de SQL Server a MongoDB sin afectar la lógica de negocio?

La sustitución de SQL Server por MongoDB sería más sencilla si la aplicación estuviera organizada mediante los principios de Clean o Hexagonal.

El procedimiento consistiría en:

1. Mantener las entidades del dominio y los casos de uso sin cambios, siempre que sus requisitos funcionales permanezcan iguales.
2. Conservar la interfaz `ProductoRepository`, que define las operaciones que necesita el núcleo de negocio.
3. Crear una implementación nueva, por ejemplo, `MongoProductoRepository`, que utilice las herramientas y consultas propias de MongoDB.
4. Configurar la aplicación para utilizar el nuevo adaptador en lugar del adaptador de SQL Server.
5. Ejecutar las pruebas unitarias y de integración para verificar que la nueva implementación respeta el contrato y conserva el comportamiento esperado.

La migración no es necesariamente automática: pueden existir diferencias entre el modelo relacional de SQL Server y el modelo documental de MongoDB, así como requisitos de migración de datos, consultas y transacciones. Sin embargo, mantener aislada la persistencia evita que esas diferencias obliguen a modificar las reglas de negocio, siempre que las capacidades requeridas puedan conservarse mediante el nuevo adaptador.

El desacoplamiento no implica que cualquier sistema gestor de bases de datos pueda sustituirse sin esfuerzo. El contrato de ‘ProductoRepository‘ debe expresar las operaciones que necesita el dominio y evitar exponer tipos, consultas o mecanismos específicos de SQL Server. Si MongoDB no permite satisfacer alguna operación con la misma semántica o comportamiento, será necesario adaptar la implementación e incluso revisar determinadas decisiones del modelo de persistencia.

La arquitectura reduce la propagación de los cambios tecnológicos, pero no elimina los problemas derivados de diferencias reales entre los modelos de datos.

1.6. Conclusión

El principal problema del sistema de inventario es el acoplamiento entre la lógica de negocio y los detalles de infraestructura. La arquitectura tradicional de N-Capas puede ser sencilla de comprender y desarrollar inicialmente, pero, si sus dependencias no se controlan adecuadamente, puede convertirse en un sistema rígido, difícil de probar y costoso de modificar.

La Arquitectura Limpia aborda este problema mediante la Regla de Dependencia, que exige que el código fuente apunte hacia el núcleo de negocio. La Arquitectura Hexagonal lo aborda mediante puertos y adaptadores, que permiten conectar el núcleo con diferentes tecnologías sin acoplarlo a ellas.

Ambas alternativas mejoran la testabilidad, la mantenibilidad y la capacidad de evolución del sistema. En particular, facilitan la sustitución de SQL Server por MongoDB al permitir que el cambio se concentre en la infraestructura, preservando las reglas de negocio.

No obstante, estas ventajas tienen un coste: definir interfaces, implementar adaptadores y mantener una estructura más elaborada requiere tiempo y esfuerzo adicionales. Por eso, la decisión arquitectónica debe considerar la complejidad del sistema y sus necesidades futuras. Para un sistema de inventario que evoluciona y necesita pruebas fiables, la inversión en desacoplamiento puede compensar ampliamente ese coste inicial.

2. Comparación de Estilos: La Simetría del Hexágono vs. la Jerarquía de las Capas

La Arquitectura Hexagonal de Cockburn y la Arquitectura Onion de Palermo comparten el objetivo de aislar el núcleo de negocio, pero utilizan representaciones visuales y metáforas radicalmente diferentes: un hexágono simétrico frente a capas concéntricas.

Analice en profundidad qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir de forma intuitiva. ¿Cómo se traduce el concepto de "Puerto y Adaptador" en la práctica de definir interfaces en las capas internas de la Arquitectura Onion? ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

Respuesta

2.1. Análisis conceptual: simetría hexagonal frente a capas

Las arquitecturas por capas y la Arquitectura Onion emplean representaciones diferentes que ayudan a organizar las responsabilidades del sistema. La arquitectura N-Capas suele representarse mediante capas apiladas, mientras que Onion utiliza capas concéntricas alrededor del modelo de dominio. Aunque ambas permiten separar responsabilidades, una representación apilada puede inducir a pensar en una secuencia espacial rígida entre presentación, negocio y datos.

La Arquitectura Hexagonal, propuesta por Alistair Cockburn, destaca un aspecto distinto: el núcleo de la aplicación se comunica con el exterior a través de puertos y adaptadores. Su representación no pretende establecer que la interfaz de usuario, la base de datos o los servicios externos sean funcionalmente idénticos, sino mostrar que todos se encuentran fuera del núcleo y que sus interacciones deben estar delimitadas mediante contratos explícitos (Cockburn 2005).

Esta simetría permite representar diversas formas de entrada y salida sin imponer una posición superior o inferior a determinados componentes. De este modo, el sistema puede incorporar distintos mecanismos de interacción manteniendo la lógica de negocio independiente de sus detalles tecnológicos.

2.2. Traducción de «Puerto y Adaptador» en la práctica de definir interfaces en las capas internas de la Arquitectura Onion

En la **Arquitectura Onion** (Palermo, 2008), el centro absoluto es el *Modelo de Dominio*, rodeado por capas concéntricas de servicios de dominio, servicios de aplicación y capas externas de infraestructura. El concepto de «Puerto y Adaptador» se traduce directamente en la práctica mediante el uso estricto del **Principio de Inversión de Dependencias (DIP)** de Robert C. Martin (2017):

- **Definición de Puertos (Interfaces en las capas internas):** En las capas internas (dominio y aplicación), el sistema define interfaces (contratos) que declaran qué operaciones necesita realizar para interactuar con el exterior (por ejemplo, una interfaz para persistencia

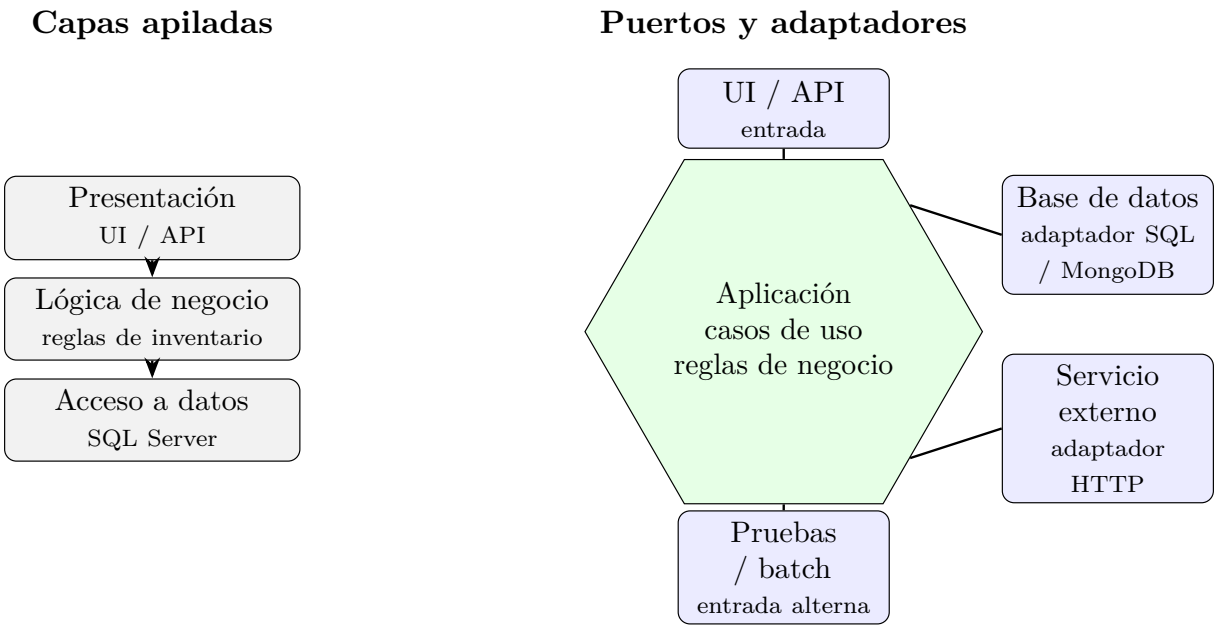


Figura 4: Comparación conceptual entre capas apiladas y Arquitectura Hexagonal. Esta última destaca el límite entre el núcleo y múltiples interacciones externas, sin imponer una posición superior o inferior a la interfaz de usuario o a la base de datos.

o pasarelas de pago). Estas interfaces residen en el núcleo y pertenecen conceptualmente al dominio, sin conocer ningún detalle tecnológico ni librerías específicas.

- **Implementación de Adaptadores (Capas externas):** Las capas externas de infraestructura implementan dichos puertos. Por ejemplo, un adaptador de persistencia implementa la interfaz utilizando un ORM o un motor de base de datos específico (como SQL Server o MongoDB).
- **Flujo de dependencias:** De este modo, las dependencias del código fuente siempre apuntan hacia adentro, permitiendo que el núcleo permanezca completamente desacoplado de las tecnologías de infraestructura.

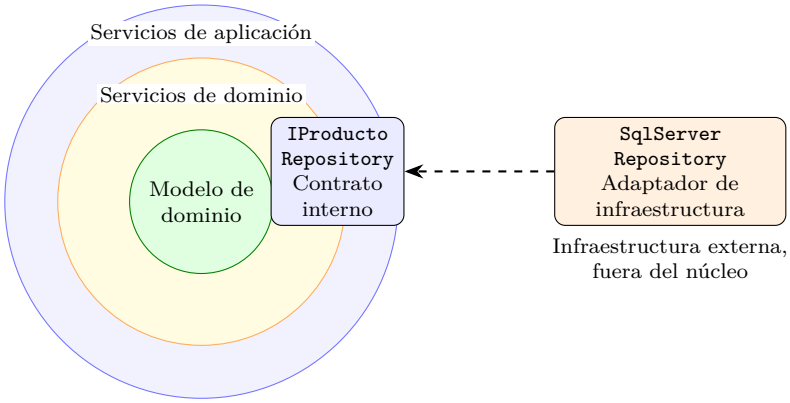


Figura 5: En Onion, las capas internas definen los contratos necesarios para el funcionamiento de la aplicación; la infraestructura externa proporciona sus implementaciones concretas.

2.3. El verdadero propósito del hexágono: Eliminar la asimetría izquierda-derecha o arriba-abajo

El hexágono no posee seis lados porque la arquitectura requiera exactamente seis puertos. Su forma constituye un recurso visual para evitar las limitaciones de una representación estrictamente lineal y hacer visible que la aplicación puede interactuar con múltiples elementos externos (Cockburn 2005).

El número de puertos depende de los propósitos de interacción que el sistema necesite delimitar. Además, un mismo puerto puede admitir diferentes adaptadores, siempre que respeten el contrato correspondiente. Por ejemplo, una interfaz de persistencia puede ser implementada mediante SQL Server, MongoDB o una implementación falsa para pruebas. La forma geométrica no determina el número de interfaces ni de tecnologías compatibles con ellas.

Las razones principales de esta elección geométrica y metafórica son:

- **Eliminar la asimetría espacial:** Los diagramas en línea recta o en pila vertical sugieren un flujo unidireccional rígido y otorgan privilegios espaciales artificiales a ciertos componentes (haciendo que parezca que la base de datos está al final de un proceso lineal).
- **Espacio para múltiples interfaces:** Un hexágono proporciona una figura simétrica con múltiples caras perimetrales que permite ilustrar visualmente que el núcleo de la aplicación puede ser abordado desde distintas direcciones simultáneamente (interfaces de usuario, pruebas automatizadas, servicios batch, persistencia).
- **Equivalencia de canales:** Visualmente, ningún lado del hexágono es superior o inferior, reforzando el principio de que el interior no sabe nada del exterior y que cualquier adaptador puede ser reemplazado o añadido sin alterar la lógica de negocio central.

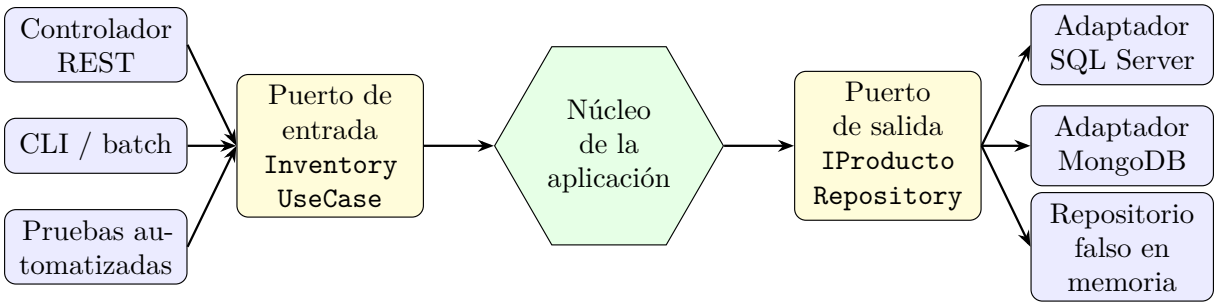


Figura 6: Dos puertos con múltiples adaptadores posibles. Los puertos se definen por el propósito de la interacción, no por el número de lados del hexágono; varios adaptadores pueden satisfacer un mismo puerto. Los adaptadores de persistencia son alternativas, no tienen que utilizarse todos simultáneamente.

3. Aplicación de DDD y Bounded Context en un Dominio Complejo

Una empresa de seguros necesita desarrollar un sistema para gestionar pólizas de automóviles. El sistema debe interactuar con un departamento de siniestros, un sistema de facturación de primas y una aplicación para que los ajustadores de seguros evalúen daños en el lugar del accidente. Cada uno de estos subsistemas tiene su propia terminología y preocupaciones. Por ejemplo, "póliza" para un actuario es un contrato con cláusulas, para facturación es una fuente de ingresos y para un ajustador es un número de referencia.

Basado en los conceptos de Domain-Driven Design (Evans) y Bounded Context (Richards y Ford), explique cómo dividiría este sistema en varios contextos delimitados. ¿Cómo evitaría que el modelo de "Póliza" se contamine con detalles de siniestros o facturación? ¿Cómo se comunicarían estos contextos de manera desacoplada?

Relacione esta solución con la implementación de una Arquitectura Orientada al Dominio (como Onion o Clean) y la creación de un Adaptador de Servicio para la comunicación externa, tal como se describe en la Guía de Arquitectura N-Capas.

Respuesta.

En el desarrollo de sistemas de software empresariales de alta complejidad, como una plataforma de seguros de automóviles, uno de los errores más críticos es intentar construir un modelo de dominio unificado y monolítico. Como señalan Richards y Ford (2020), conceptos centrales del negocio como el término "póliza" adquieren significados, reglas y responsabilidades radicalmente opuestas según el departamento que los manipule. Para resolver este desafío, **Domain-Driven Design (DDD)** (Evans, 2003) introduce el concepto de **Bounded Context (Contexto Delimitado)**, el cual delimita explícitamente la aplicabilidad de un modelo de dominio para asegurar que su Lenguaje Ubicuo sea coherente, inequívoco y autónomo en su interior.

3.1. Descomposición del sistema en contextos delimitados

Para estructurar el sistema de seguros de manera desacoplada, se definen cuatro contextos delimitados principales, cada uno con su propia semántica y conjunto de responsabilidades:

1. **Contexto de Suscripción y Gestión de Pólizas:** Enfocado en la evaluación de riesgos, emisión y administración del contrato legal. En este contexto, una "póliza" es un acuerdo formal que agrupa cláusulas, coberturas, vigencias y sumas aseguradas.

2. **Contexto de Facturación de Primas:** Enfocado en la gestión financiera. Aquí, la "póliza" se reduce a una fuente de ingresos, un identificador financiero ligado a un plan de pagos, cuotas, vencimientos y estados de cuenta.

3. **Contexto de Siniestros:** Orientado a la gestión de reclamaciones e indemnizaciones. Para este módulo, la póliza representa el marco jurídico-financiero bajo el cual se evalúa la validez de un incidente reportado.

4. **Contexto de Evaluación de Daños (Ajustadores):** Diseñado para la aplicación móvil de campo. El ajustador no requiere conocer las cláusulas del actuario ni las deudas de facturación; para él, la póliza es simplemente un número de referencia o identificador rápido que valida la preexistencia de cobertura en el lugar del accidente.

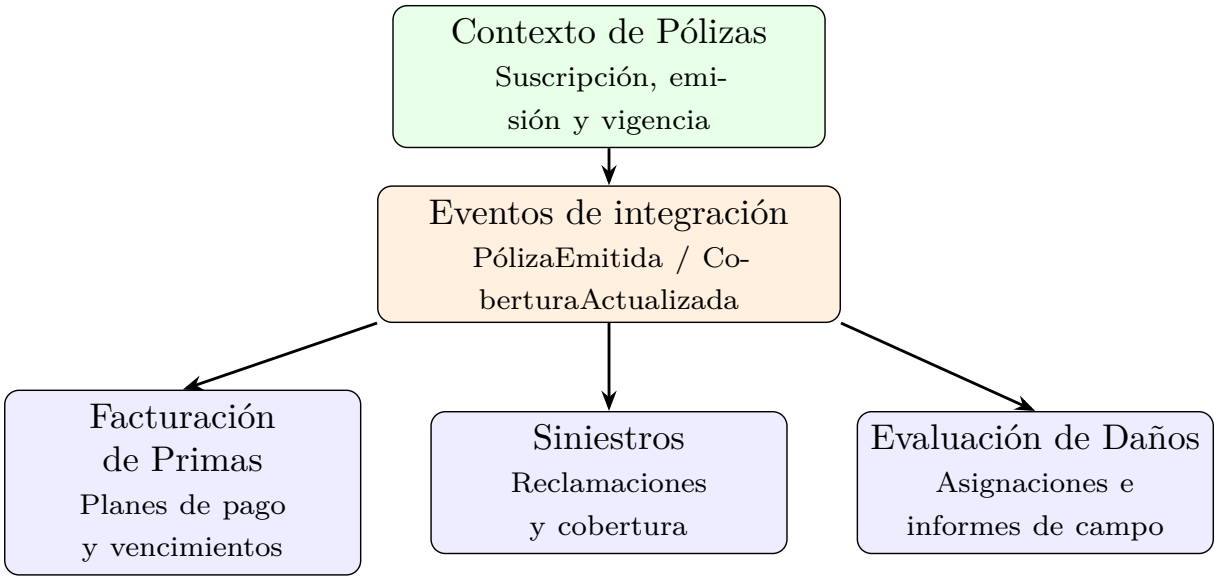


Figura 7: Mapa conceptual de los cuatro contextos delimitados del sistema de seguros y una propuesta de comunicación mediante eventos de integración. Cada consumidor interpreta los contratos recibidos según sus propias responsabilidades.

3.2. Prevención de la contaminación del modelo de dominio

Compartir una única entidad de dominio denominada 'Poliza' entre Suscripción, Facturación, Siniestros y Evaluación de Daños introduciría un acoplamiento semántico entre modelos que responden a necesidades diferentes. Una modificación de las reglas contractuales podría propagarse hacia la facturación o la evaluación de daños, aunque estos contextos no necesitaran conocer dicho cambio. Para evitarlo, cada Bounded Context debe mantener su propio modelo de dominio y controlar cómo representa y persiste la información relevante para sus responsabilidades, por lo que se aplican los siguientes principios arquitectónicos:

- **Modelos de Dominio Independientes:** Cada Bounded Context posee su propio modelo de objetos en memoria y su propio esquema de persistencia (o tablas dedicadas).
- **Capas de Traducción (*Anti-Corruption Layer - ACL*):** Cuando un contexto necesita interactuar con otro, se implementan traductores explícitos que transforman los conceptos del lenguaje emisor al lenguaje receptor, evitando que los detalles internos de un subsistema contaminen las reglas de negocio de otro.

3.3. Comunicación desacoplada entre contextos

Para evitar llamadas síncronas fuertemente acopladas que degraden la disponibilidad del sistema, la comunicación entre los contextos delimitados se resuelve mediante un estilo arquitectónico guiado por eventos:

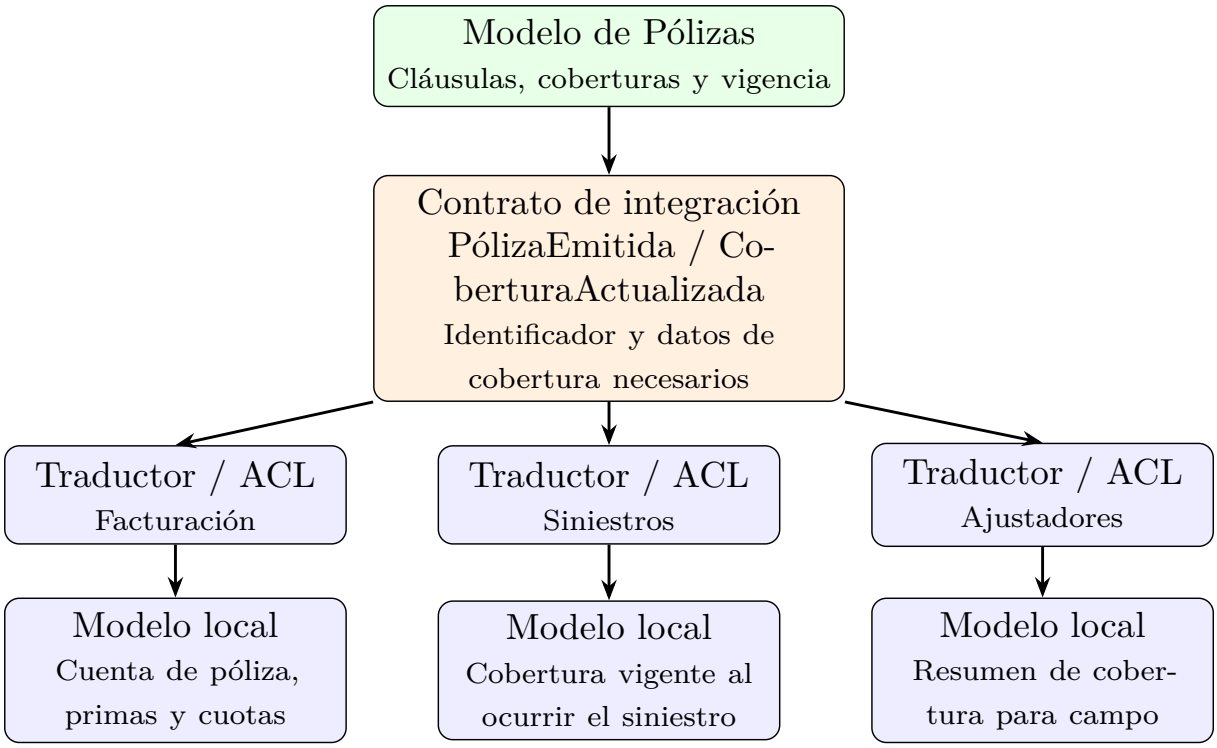


Figura 8: Protección de los modelos de dominio mediante contratos de integración y capas anticorrupción. Cada contexto traduce los datos recibidos a su propio modelo en lugar de compartir la entidad de dominio del contexto de Pólizas.

- **Eventos de Dominio (*Domain Events*):** Cuando ocurre un cambio de estado relevante en el núcleo de un contexto (por ejemplo, se emite una póliza o se declara un siniestro), este emite un evento asíncrono inmutable.
- **Publicación y Suscripción (*Pub/Sub*):** Los contextos interesados (como Facturación) se suscriben de forma independiente a dichos eventos para actualizar sus propios registros de manera desacoplada temporal y espacialmente, eliminando las dependencias directas de código fuente entre los módulos.

3.4. Alineación con arquitecturas orientadas al dominio y adaptadores de servicio

Esta partición por contextos delimitados se implementa de manera natural utilizando arquitecturas basadas en capas concéntricas o simétricas, como la **Arquitectura Onion** (Palermo, 2008) o la **Arquitectura Limpia** (Martin, 2017):

- **Protección del Núcleo:** Cada Bounded Context aísla su lógica de negocio pura en el centro de la arquitectura (Entidades y Casos de Uso), blindándola frente a tecnologías externas, frameworks o bases de datos.
- **Adaptadores de Servicio (*Service Adapters*):** Tal como se describe en la *Guía de Arquitectura N-Capas orientada al Dominio con .NET* (De la Torre Llorente et al. 2010)), la interacción con servicios externos (como pasarelas de pago de terceros o la API de los ajustadores de campo) no debe penetrar el núcleo del dominio. Para ello, se diseñan

Adaptadores de Servicio en la capa de infraestructura o en el exterior del hexágono. Estos adaptadores actúan como implementaciones de puertos definidos internamente, encargándose de serializar peticiones, traducir protocolos HTTP o transformar formatos de datos externos, garantizando así el cumplimiento estricto de la Regla de Dependencia y una alta mantenibilidad a largo plazo.

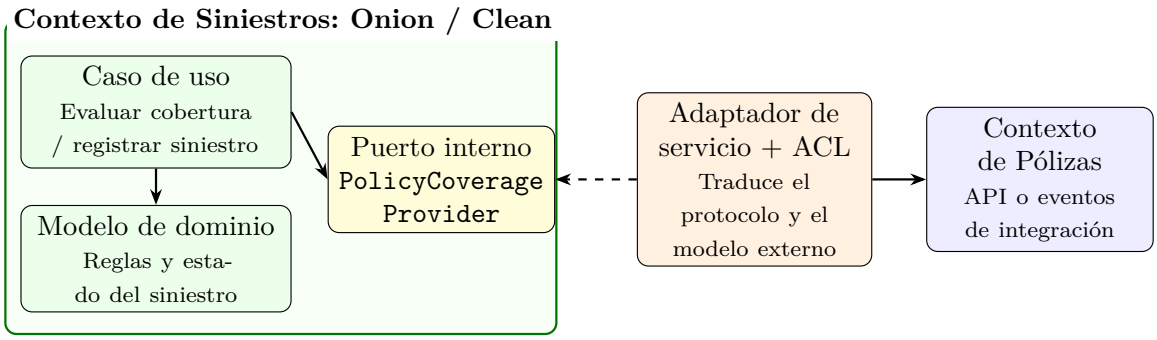


Figura 9: Relación entre el núcleo de un contexto delimitado y sus adaptadores externos. El caso de uso depende del puerto interno, mientras que el adaptador conecta ese contrato con la API o los eventos del contexto de Pólizas.

4. Evaluación de la Inversión de Dependencias en la Práctica

Un equipo de desarrollo está trabajando en un sistema de gestión de pedidos. La lógica de negocio principal se encuentra en una clase `OrderProcessor` que, para completar su tarea, necesita comunicarse con un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa: `OrderProcessor` instancia y llama directamente a la clase `AddressValidatorService`, que a su vez depende de una biblioteca HTTP específica para comunicarse con el servicio externo.

Con base en los principios de la Inversión de Dependencias (Martin) y la Regla de Dependencia (Martin), analice y responda:

(a) ¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? Explique específicamente qué dirección deberían tener las dependencias del código fuente y por qué la implementación actual no la cumple.

(b) Rediseñe la interacción entre `OrderProcessor` y `AddressValidatorService` para que se alinee con los principios de estas arquitecturas. Defina claramente qué componente (interfaz) debe residir en el núcleo de la aplicación y qué componente debe residir en la capa de infraestructura o en el exterior del hexágono. Justifique su diseño basándose en el principio de que "los detalles deben depender de las abstracciones".

(c) Explique cómo este nuevo diseño permite que la lógica de negocio (`OrderProcessor`) sea probada de forma aislada, sin necesidad de realizar llamadas HTTP reales al servicio externo y sin depender de la biblioteca HTTP subyacente. ¿Qué implicaciones tiene este diseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro?

Respuesta

En el escenario planteado, la clase `OrderProcessor` contiene la lógica principal para procesar pedidos y necesita validar las direcciones mediante un servicio externo. Sin embargo, su implementación actual instancia directamente a `AddressValidatorService`, que depende de una biblioteca HTTP específica.

El problema fundamental no es que el sistema utilice HTTP, sino que la lógica de negocio depende directamente de un detalle de infraestructura. Esta relación dificulta las pruebas unitarias, aumenta el acoplamiento y hace que un cambio tecnológico pueda obligar a modificar componentes que deberían permanecer estables.

Desde la perspectiva de Richards y Ford (2020), la modularidad y el control del acoplamiento son aspectos esenciales del diseño arquitectónico, pues determinan cómo afectan los cambios de un componente al resto del sistema. La solución consiste en aplicar el Principio de Inversión de Dependencias (*Dependency Inversion Principle*, DIP) y respetar la Regla de Dependencia (*Dependency Rule*) de Martin (2017).

4.1. a) ¿Por qué la dependencia directa viola los principios de estas arquitecturas?

4.1.1. Análisis del diseño actual

La estructura original implica dos responsabilidades diferentes:

- `OrderProcessor`: contiene las reglas y el flujo de procesamiento de pedidos.
- `AddressValidatorService`: conoce los detalles de comunicación con el servicio externo, incluida la biblioteca HTTP utilizada.

La dificultad aparece cuando `OrderProcessor` conoce directamente la clase concreta `AddressValidatorService`. Al crearla mediante una instrucción como `new AddressValidatorService()`, la lógica de negocio queda vinculada a su implementación específica.

Si el equipo decide sustituir la biblioteca HTTP por otra, modificar el protocolo de comunicación o cambiar el proveedor de validación, ese cambio tecnológico puede propagarse hacia las clases que dependen de la implementación actual.

Esta situación constituye un caso de acoplamiento entre políticas de negocio y mecanismos de infraestructura. Las políticas expresan qué debe hacer el sistema; los mecanismos determinan cómo se ejecutan determinadas operaciones técnicas. Una arquitectura mantenible procura que los segundos puedan cambiar sin alterar innecesariamente las primeras.

4.1.2. Violación del Principio de Inversión de Dependencias (DIP)

El Principio de Inversión de Dependencias establece que:

- Los módulos de alto nivel no deben depender directamente de módulos de bajo nivel.
- Ambos deben depender de abstracciones.
- Las abstracciones no deben depender de los detalles de implementación; los detalles deben depender de las abstracciones.

En el caso analizado, `OrderProcessor` representa un módulo de alto nivel, porque coordina una operación del negocio. `AddressValidatorService`, junto con la biblioteca HTTP y sus mecanismos de comunicación, representa el detalle técnico utilizado para cumplir una parte de esa operación. La relación actual incumple el principio porque el módulo de alto nivel depende de una implementación concreta de bajo nivel.

Para corregirlo, debe introducirse una abstracción que represente la capacidad que necesita el negocio: validar una dirección. La lógica de procesamiento dependerá de ese contrato, no de una implementación HTTP determinada (Martin 2017).

4.1.3. Violación de la Regla de Dependencia

La Regla de Dependencia, expuesta por Martin (2017) en *Clean Architecture*, establece que las dependencias del código fuente deben apuntar hacia las capas internas, donde se encuentran las

políticas de mayor nivel. Esto significa que el núcleo de la aplicación no debe importar ni conocer las clases, interfaces o formatos propios de las capas externas.

La implementación original incumple esta regla porque `OrderProcessor` necesita conocer la clase concreta que implementa la comunicación externa. La Regla de Dependencia y el DIP están relacionados, pero no son idénticos:

Principio	Aplicación al caso
Inversión de Dependencias (DIP)	Sustituir la dependencia directa de una implementación por una dependencia de una abstracción.
Regla de Dependencia	Garantizar que las dependencias del código fuente apunten hacia el núcleo de la aplicación, no hacia la infraestructura.

Cuadro 1: Comparativa entre el DIP y la Regla de Dependencia.

4.1.4. Relación con las arquitecturas Hexagonal, Limpia y Onion

Las tres arquitecturas permiten resolver este problema mediante límites explícitos entre el núcleo y los mecanismos externos:

- **Arquitectura Hexagonal (Puertos y Adaptadores):** La aplicación define un puerto de salida que expresa la operación requerida para validar una dirección. Un adaptador de infraestructura implementa ese puerto utilizando la biblioteca HTTP y el proveedor externo (Cockburn 2005).
- **Arquitectura Limpia (*Clean Architecture*):** `OrderProcessor` pertenece al ámbito de los casos de uso o de la lógica de aplicación. La interfaz que necesita se define en una capa interna, mientras que la implementación HTTP reside en una capa externa (Martin 2017).
- **Arquitectura Onion (Cebolla):** La abstracción necesaria para validar direcciones se sitúa en una capa interna de aplicación, mientras que la infraestructura proporciona una implementación concreta (Palermo 2008).

En síntesis, las tres propuestas coinciden en una decisión esencial: la infraestructura se adapta a las necesidades del núcleo; el núcleo no se estructura alrededor de la infraestructura.

4.2. b) Rediseño de la interacción entre `OrderProcessor` y `AddressValidator Service`

4.2.1. Definición de la solución

Se propone introducir una interfaz denominada `AddressValidationPort`, que representa el contrato de validación requerido por la aplicación. Los componentes quedarían distribuidos de la siguiente manera:

4.2.2. Diagrama de la arquitectura propuesta

El diagrama distingue dos conceptos clave:

Componente	Ubicación	Responsabilidad
`OrderProcessor`	Núcleo de aplicación	Coordinar el procesamiento de pedidos y aplicar las reglas correspondientes.
`AddressValidationPort`	Núcleo de aplicación	Definir el contrato para validar una dirección.
`HttpAddressValidation Adapter`	Infraestructura	Implementar el contrato y comunicarse con el proveedor externo mediante HTTP.
`Biblioteca HTTP`	Infraestructura	Ejecutar las solicitudes y gestionar los detalles del protocolo de comunicación.
`Composition Root`	Configuración	Crear la implementación concreta e inyectarla en `OrderProcessor`.

Cuadro 2: Distribución arquitectónica de los nuevos componentes.

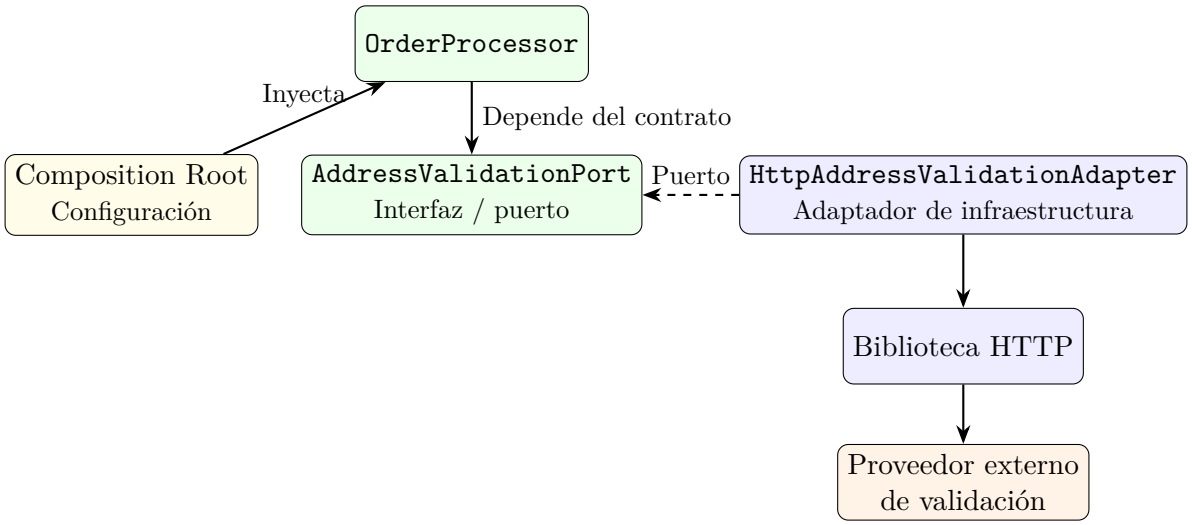


Figura 10: Inversión de dependencias y flujo de control mediante puertos y adaptadores.

- **Dependencia del código fuente:** el adaptador de infraestructura depende de la interfaz definida por la aplicación.
- **Flujo de ejecución:** cuando `OrderProcessor` solicita una validación, la ejecución llega al adaptador concreto mediante polimorfismo.

4.2.3. Ejemplo de implementación

A continuación se muestra una implementación en Python que ilustra el diseño propuesto.

1. Contrato en el núcleo de la aplicación:

Listing 1: Definición del puerto de validación.

```python
from typing import Protocol

class AddressValidationPort(Protocol):
    def is_valid(self, address: str) -> bool:
        ...
```

2. Lógica de negocio dependiente del contrato (Inyección de Dependencias):

Listing 2: Inyección del puerto en el procesador de pedidos.

```python
class OrderProcessor:
    def __init__(self, address_validator: AddressValidationPort):
        self._address_validator = address_validator

    def process(self, order):
        if not self._address_validator.is_valid(order.shipping_address):
            raise ValueError("La direcci\u00f3n de entrega no es v\u00e1lida")

        # Continuar con las reglas de procesamiento del pedido.
        return "Pedido procesado"
```

3. Adaptador concreto en la capa de infraestructura:

Listing 3: Adaptador HTTP en la capa de infraestructura.

```python
class HttpAddressValidationAdapter:
    def __init__(self, http_client, endpoint: str):
        self._http_client = http_client
        self._endpoint = endpoint

    def is_valid(self, address: str) -> bool:
        response = self._http_client.post(
            self._endpoint,
            json={"address": address},
        )
        response.raise_for_status()
        return response.json()["valid"]
```

4.3. c) Efectos sobre las pruebas, la mantenibilidad y el cambio de proveedor

4.3.1. Pruebas unitarias aisladas

Con la nueva arquitectura, las pruebas unitarias pueden utilizar un objeto simulado (*mock*) o una implementación falsa (*fake*) que cumpla el contrato de `AddressValidationPort`:

Listing 4: Implementación falsa para pruebas unitarias.

```
class FakeAddressValidator:
    def __init__(self, valid: bool):
        self.valid = valid

    def is_valid(self, address: str) -> bool:
        return self.valid
```

Esto permite verificar los diferentes comportamientos de la lógica de negocio sin realizar peticiones HTTP reales:

Listing 5: Pruebas unitarias sobre OrderProcessor.

```
# Caso 1: La direccion es valida
processor = OrderProcessor(FakeAddressValidator(valid=True))
assert processor.process(order) == "Pedido procesado"

# Caso 2: La direccion no es valida
processor = OrderProcessor(FakeAddressValidator(valid=False))
try:
    processor.process(order)
    assert False, "Se esperaba rechazar el pedido"
except ValueError:
    pass
```

Distinguimos dos niveles de pruebas:

- **Pruebas unitarias:** verifican el comportamiento de `OrderProcessor` usando objetos simulados o falsos.
- **Pruebas de integración:** verifican por separado que `HttpAddressValidationAdapter` se comunica correctamente con la API externa.

4.3.2. Mantenibilidad y evolución tecnológica

La nueva estructura reduce el alcance de los cambios tecnológicos. Si se decide cambiar el proveedor de validación, basta con crear una nueva clase `NewProviderAddressValidationAdapter` que implemente el mismo puerto.

4.3.3. Implicaciones para el futuro

El diseño propuesto aporta tres beneficios arquitectónicos principales:

Escenario de cambio	**Diseño con dependencia directa**	**Diseño con inversión de dependencias**
Sustituir biblioteca HTTP	Puede obligar a modificar componentes acoplados a ella.	Principalmente afecta al adaptador HTTP.
Cambiar proveedor externo	Puede propagar cambios a la lógica de negocio.	Se crea o modifica un adaptador compatible con el mismo contrato.
Probar el procesamiento	Puede requerir preparar dependencias externas.	Se utilizan implementaciones simuladas o falsas.
Evolucionar el núcleo	Los detalles técnicos pueden condicionar modificaciones.	El núcleo conserva mayor independencia tecnológica.

Cuadro 3: Comparativa de impacto ante cambios tecnológicos.

1. **Mayor testabilidad:** las reglas de procesamiento se comprueban sin depender de servicios externos.
2. **Mayor mantenibilidad:** los cambios en la comunicación externa se concentran en una frontera específica.
3. **Mayor flexibilidad tecnológica:** el sistema puede incorporar otro proveedor mediante una nueva implementación del puerto.

Conclusión

La solución consiste en definir `AddressValidationPort` dentro del núcleo de aplicación, hacer que `OrderProcessor` dependa de esa abstracción e implementar el contrato mediante `HttpAddressValidationAdapter` en infraestructura. Este rediseño aplica los principios de la Arquitectura Hexagonal, Limpia y Onion, logrando que la aplicación dependa de lo que necesita hacer y no de la tecnología utilizada para hacerlo.

5. Análisis de un Problema Real: el "Kitty Problem" y el Principio de Abierto/Cerrado

En su libro *Fundamentals of Software Architecture*, Richards y Ford presentan el "Kitty Problem" para ilustrar un fallo común en el diseño de arquitecturas orientadas a servicios. Describa este problema: un requisito transversal, como la entrega de gatitos, afecta a múltiples servicios previamente existentes (`TaxiUI`, `TaxiFinder`, `TaxiSelector`, etc.), requiriendo cambios coordinados en todos ellos.

¿Por qué la solución de dividir el sistema en servicios basados en verbos o entidades funcionales, como `TaxiFinder` o `TaxiSelector`, llevó a este problema? Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio?

Proponga una solución utilizando componentes como `Kittens` y `Rides` que se añadan como plugins, explicando cómo esto respeta el Principio de Abierto/Cerrado (OCP) y qué papel juegan los Adaptadores en esta extensión.

Respuesta

El *Kitty Problem* es un caso de estudio clásico planteado en la literatura de arquitectura de software (Richards & Ford; Robert C. Martin) que ilustra los peligros de descomponer un sistema basándose en acciones o pasos de flujo en lugar de dominios de negocio.

En el sistema original de agregación de taxis, la arquitectura se estructuró en microservicios por verbos/tareas:

- `TaxiUI`: Interfaz de usuario para realizar solicitudes.
- `TaxiFinder`: Servicio encargado de buscar taxis disponibles.
- `TaxiSelector`: Servicio para filtrar y seleccionar la mejor opción de transporte.
- `TaxiDispatcher`: Servicio para asignar y despachar el vehículo al cliente.

Cuando el departamento comercial decidió lanzar una nueva línea de negocio llamada **entrega de gatitos** (*Kitten Delivery Service*), la nueva funcionalidad introdujo requisitos transversales:

1. Verificar alergias a felinos tanto en el conductor como en el cliente.
2. Seleccionar puntos de recogida específicos para las mascotas.
3. Requerir tiempos y procesos de desinfección/limpieza especial para los vehículos utilizados.

El problema: Como el sistema estaba dividido horizontalmente por funciones o verbos (`Finder`, `Selector`, `Dispatcher`), la implementación de este nuevo requisito obligó a modificar, probar y desplegar nuevamente **todos y cada uno de los microservicios del sistema**, destruyendo la autonomía y el desacoplamiento prometidos por la arquitectura de microservicios.

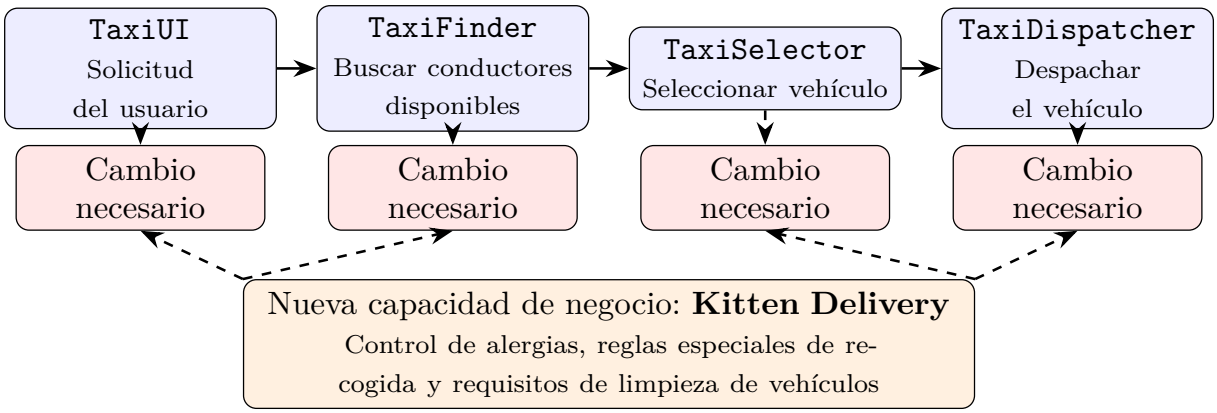


Figura 11: El Kitty Problem: una nueva capacidad de negocio atraviesa servicios organizados por etapas del flujo de trabajo.

5.1. Análisis de la Causa Raíz

La causa raíz de esta fragilidad arquitectónica radica en el **particionamiento funcional por verbos** (*Functional Partitioning*).

Las funcionalidades del negocio son por naturaleza **transversales** (*cross-cutting concerns*). Cuando un sistema se divide por etapas del flujo de trabajo, cualquier nueva característica de negocio atraviesa horizontalmente todas las etapas. Como consecuencia:

- Se genera un acoplamiento fuerte entre servicios supuestamente independientes.
- Se violan los principios de despliegue independiente.
- Cambiar una regla de negocio requiere modificaciones coordinadas en múltiples repositorios.

5.2. Rediseño basado en DDD y Clean Architecture

Para evitar que una nueva característica afecte a los componentes existentes, el sistema debe reorganizarse mediante un **particionamiento por dominio** (*Domain Partitioning*) y la aplicación de los principios de Clean Architecture:

5.2.1. Abstracción del Núcleo

Los servicios operativos base (`TaxiFinder`, `TaxiSelector`, `TaxiDispatcher`) dejan de ser servicios concretos y se convierten en **interfaces o clases base abstractas** en el núcleo del dominio.

5.2.2. Componentes Verticales por Dominio

La lógica de negocio se divide en módulos o paquetes verticales independientes:

- Módulo `Rides`: Encapsula la lógica del transporte de pasajeros convencional.
- Módulo `Kittens`: Encapsula las reglas específicas del transporte de mascotas.

5.2.3. Patrones de Diseño

- **Patrón Strategy / Template Method:** Las clases dentro de `Kittens` y `Rides` extienden o implementan las abstracciones del núcleo, encapsulando sus diferencias (por ejemplo, el filtro de alergias en la selección).
- **Inversión de Dependencias (DIP):** Las capas externas dependen de las abstracciones centrales; el núcleo no conoce la existencia ni de `Rides` ni de `Kittens`.

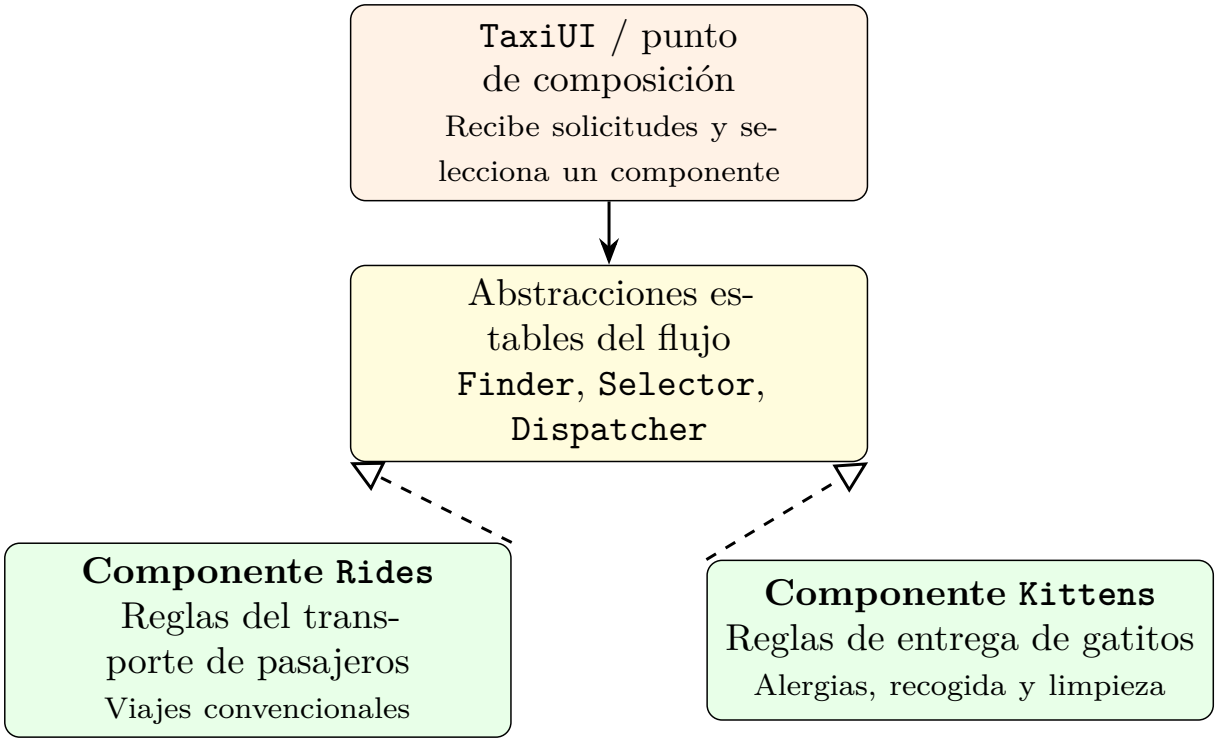


Figura 12: Rediseño mediante componentes verticales que implementan abstracciones comunes. El sistema puede seleccionar el comportamiento adecuado sin distribuir todas las reglas de cada capacidad entre los servicios del flujo de trabajo.

5.3. Cumplimiento del Principio de Abierto/Cerrado (OCP)

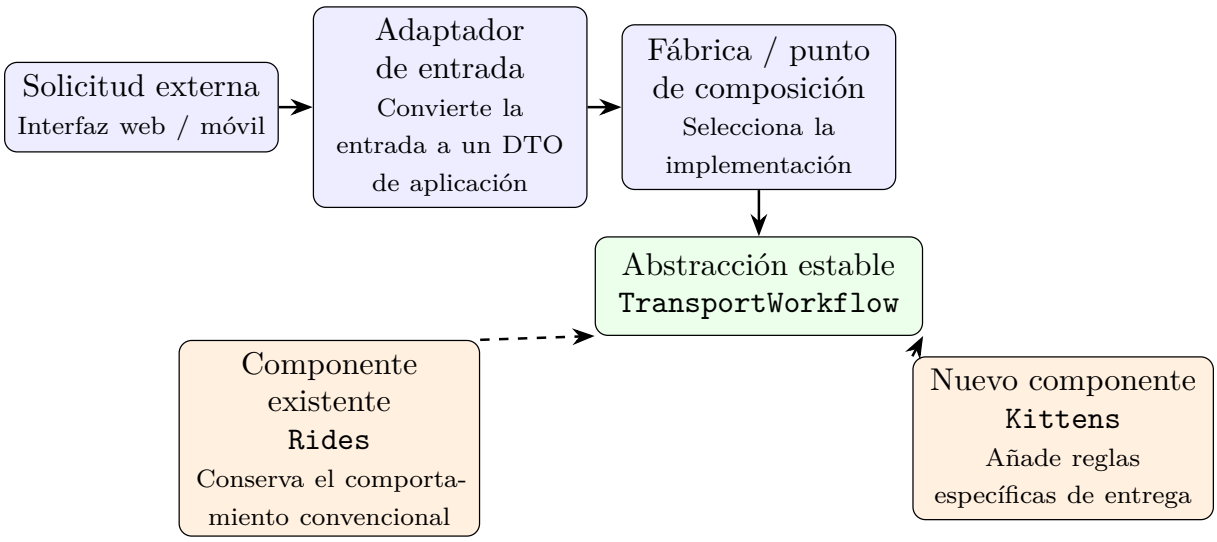
El **Principio de Abierto/Cerrado** (*Open/Closed Principle - OCP*) establece que los módulos de software deben estar *abiertos a la extensión, pero cerrados a la modificación.*

- **Cerrado a la modificación:** El núcleo abstracto del sistema y el módulo existente `Rides` no requieren ningún cambio de código cuando se añade el servicio de kittens.
- **Abierto a la extensión:** La nueva característica se añade como un paquete o biblioteca completamente nuevo (`Kittens.dll` / `Kittens.jar` / contenedor independiente) que se integra en tiempo de ejecución.

5.4. Rol de los Adaptadores y Fábricas

Para conectar la interfaz de usuario con los módulos sin violar el desacoplamiento:

- **Component Factories (Fábricas):** Instancian dinámicamente la estrategia adecuada (`Rides` o `Kittens`) según el contexto de la solicitud.
- **Adaptadores de Interfaz:** Transforman las entradas de la UI (`TaxiUI`) en DTOs que consumen los puertos y casos de uso abstractos del núcleo, garantizando un flujo limpio y desacoplado.



Los adaptadores aíslan los formatos externos; la fábrica selecciona un componente compatible y el contrato estable protege el núcleo. Puede ser necesario registrar la extensión o actualizar la interfaz.

Figura 13: Papel de los adaptadores y las fábricas en la extensión del sistema. OCP protege las políticas estables frente a cambios propios de una funcionalidad, sin garantizar que todos los puntos de integración permanezcan intactos.

Conclusión

El *Kitty Problem* demuestra que una arquitectura no es limpia ni mantenible únicamente por dividir el código en microservicios o capas. El criterio clave de diseño es organizar el sistema alrededor de **Contextos Delimitados (DDD)** y abstraer el núcleo para permitir la extensión mediante OCP y adaptadores, garantizando que añadir nuevas características al negocio sea una tarea puramente aditiva y no destructiva.

6. Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad

La Arquitectura N-Capas es criticada por su bajo rendimiento en características como "Agilidad general", "Facilidad de despliegue" y "Escalabilidad" (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (*open layers*) y técnicas como el *fast-lane reader*, se pueden mitigar estos problemas.

Analice esta afirmación. ¿Cómo afecta la introducción de capas abiertas al principio de Aislamiento de Capas y a la mantenibilidad a largo plazo? ¿Por qué la solución de capas abiertas puede llevar a un "Sinkhole Anti-pattern" y qué dice la regla del 80/20 sobre esto?

Finalmente, explique por qué, para un sistema con requisitos de alta escalabilidad y agilidad, una arquitectura como la Hexagonal o la Limpia, que se basan en los principios de DDD y desacoplamiento, puede resultar más adecuada que una Arquitectura N-Capas, incluso con optimizaciones.

Respuesta

6.1. Capas abiertas y degradación del Aislamiento de Capas

La propuesta del estudiante debe evaluarse considerando la compensación entre rendimiento y mantenibilidad que introduce la apertura de capas. En la Arquitectura N-Capas, el principio de **Aislamiento de Capas** (*Layers of Isolation*) busca limitar el impacto de los cambios mediante la separación de responsabilidades y la interacción a través de interfaces definidas Richards 2015.

En un esquema de **capas cerradas** (*closed layers*), una solicitud debe atravesar las capas intermedias según la estructura establecida. Esto permite que cada capa oculte sus detalles internos a las superiores. Por el contrario, las **capas abiertas** (*open layers*) permiten que una solicitud omita determinadas capas cuando existe una justificación arquitectónica para ello.

Una técnica como el *fast-lane reader* puede aprovechar esta posibilidad para acelerar determinadas operaciones de lectura, evitando el procesamiento innecesario de algunas capas. Sin embargo, esta optimización no implica necesariamente que toda la capa de presentación deba acceder directamente a la base de datos. El riesgo aparece cuando la ruta alternativa establece dependencias directas con el esquema de persistencia o con estructuras de datos propias de una tecnología concreta.

En ese caso, una modificación del modelo de datos puede obligar a cambiar componentes de presentación que deberían permanecer aislados de los detalles de persistencia. Se debilita así el aislamiento arquitectónico y aumenta el coste de mantenimiento Richards 2015; Richards y Ford 2020.

La figura 14 compara ambos enfoques. La ruta rápida puede reducir el trabajo necesario para una lectura, pero también crea una dependencia que debe justificarse y controlarse explícitamente.

Por tanto, abrir capas no constituye necesariamente un error: puede ser una decisión razonable ante requisitos de rendimiento concretos. La cuestión arquitectónica consiste en determinar si

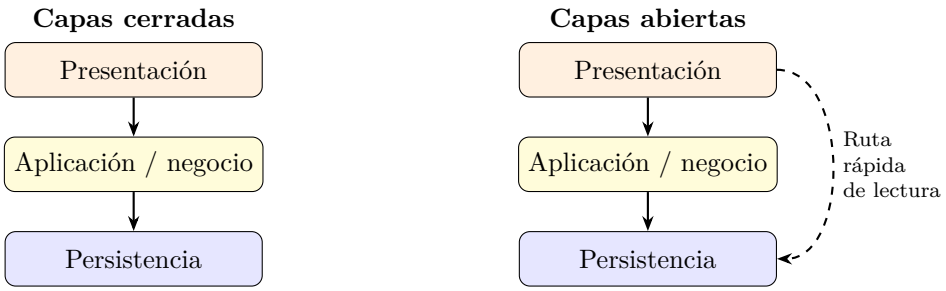


Figura 14: Comparación entre capas cerradas y abiertas. Las capas abiertas permiten rutas alternativas que pueden mejorar el rendimiento de determinadas operaciones, pero también debilitar el aislamiento entre componentes.

la mejora obtenida compensa el acoplamiento adicional y si existen mecanismos que limiten su propagación. Una excepción controlada no debe convertirse en la estructura habitual de toda la aplicación.

6.2. El Antipatrón del Sumidero (*Sinkhole Anti-pattern*) y la Regla 80/20

El **Antipatrón del Sumidero** (*Sinkhole Anti-pattern*) aparece cuando una solicitud atraviesa varias capas intermedias, pero estas se limitan a reenviar la petición sin realizar un procesamiento significativo. La arquitectura conserva entonces la complejidad de la separación en capas sin obtener todos sus beneficios funcionales Richards 2015.

Para analizar esta situación, Richards emplea la denominada **regla del 80/20** como una heurística de evaluación:

- Si aproximadamente un **20 %** de las solicitudes son operaciones de simple paso (*pass-through*), la separación en capas todavía puede estar justificada, siempre que las demás operaciones obtengan valor de la estructura arquitectónica.
- Si aproximadamente un **80 %** de las solicitudes atraviesan las capas sin procesamiento significativo, existe un indicio de que la arquitectura introduce una sobrecarga estructural que no se corresponde con las necesidades reales de la aplicación.

Esta regla no debe interpretarse como una ley matemática ni exige que cada capa ejecute lógica de negocio en todas las solicitudes. Su finalidad es orientar el análisis: si la mayoría de las operaciones únicamente atraviesa componentes intermedios, conviene revisar si todas esas capas son necesarias o si la lógica está distribuida de forma poco apropiada.

En este escenario, abrir capas puede reducir el número de pasos de determinadas solicitudes y mejorar su rendimiento. Sin embargo, si se utiliza indiscriminadamente, la optimización puede ocultar un problema de diseño más profundo. El sistema termina combinando la complejidad de una arquitectura por capas con dependencias directas que reducen su aislamiento y dificultan la evolución Richards 2015; Richards y Ford 2020.

La decisión adecuada no consiste, por tanto, en abrir todas las capas para eliminar el tránsito innecesario, sino en identificar qué operaciones requieren realmente ese procesamiento y justificar

las excepciones mediante los requisitos de calidad.

6.3. Comparación con las arquitecturas Limpia y Hexagonal

La Arquitectura N-Capas puede resultar menos apropiada para determinados requisitos de agilidad, facilidad de despliegue y escalabilidad, especialmente cuando la separación entre capas no evita el acoplamiento con los detalles técnicos. No obstante, estas limitaciones no son consecuencias inevitables de toda implementación N-Capas: dependen de cómo se definan las responsabilidades, las dependencias y las unidades de despliegue Richards 2015; Richards y Ford 2020.

Las arquitecturas **Hexagonal** y **Limpia** ofrecen mecanismos explícitos para proteger la lógica de negocio de los detalles externos (Cockburn 2005; Martin 2017). Combinadas con los principios de diseño guiado por el dominio (*Domain-Driven Design*, DDD), pueden proporcionar una base más adecuada cuando los requisitos exigen una evolución frecuente de las reglas de negocio y una mayor independencia tecnológica Evans 2003.

Sus ventajas deben entenderse en función de los siguientes factores:

1. **Inversión de dependencias y agilidad.**

   En una implementación N-Capas convencional, las capas superiores suelen utilizar los servicios de las capas inferiores. Si la lógica de negocio depende directamente de clases, consultas o estructuras específicas de persistencia, cambiar de tecnología puede obligar a modificar componentes que no deberían verse afectados.

   En la Arquitectura Limpia, la Regla de Dependencia establece que las dependencias del código fuente deben apuntar hacia las políticas de mayor nivel. La Arquitectura Hexagonal aplica un principio compatible mediante puertos definidos según las necesidades de la aplicación y adaptadores que implementan las interacciones con el exterior Martin 2017; Cockburn 2005.

   De este modo, es posible sustituir un adaptador de persistencia sin modificar necesariamente las reglas de negocio. La ventaja depende de que las abstracciones estén bien definidas y de que los detalles tecnológicos no se filtren hacia el núcleo.

2. **Modularidad y facilidad de despliegue.**

   Una aplicación organizada mediante N-Capas puede desplegarse como una única unidad, aunque también existen implementaciones con capas desplegadas por separado. De manera análoga, Clean y Hexagonal pueden utilizarse tanto en monolitos como en sistemas distribuidos.

   DDD contribuye a organizar el modelo mediante *Bounded Contexts* o contextos delimitados, que definen fronteras semánticas y de responsabilidad dentro del dominio Evans 2003. Estas fronteras facilitan la separación de modelos y la evolución de las funcionalidades, pero no implican automáticamente que cada contexto sea un servicio independiente.

   Para obtener despliegues independientes es necesario definir unidades desplegables, contratos de comunicación y mecanismos operacionales apropiados. El desacoplamiento lógico facilita

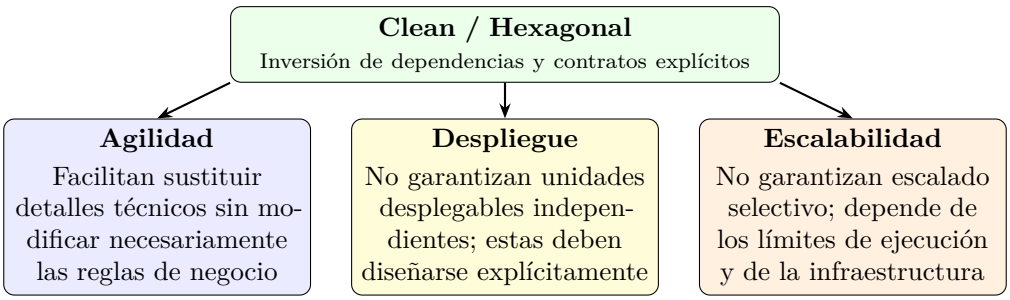
esas decisiones, pero no las sustituye.

3. **Escalabilidad y distribución de la carga.**

   Cuando una aplicación se despliega como una única unidad, una necesidad elevada de capacidad en una funcionalidad puede obligar a replicar componentes que no requieren el mismo nivel de recursos. Esto puede aumentar el coste de infraestructura y dificultar el escalado selectivo.

   La inversión de dependencias y los puertos y adaptadores facilitan aislar las reglas de negocio y sustituir mecanismos externos, pero no garantizan por sí solos escalabilidad horizontal ni ejecución independiente.

   Para escalar determinadas funcionalidades de forma selectiva pueden ser necesarias unidades de despliegue independientes, colas de mensajes, procesamiento asíncrono, cachés o separación de las operaciones de lectura y escritura. Estas técnicas responden a requisitos concretos y deben evaluarse según la carga, la latencia y los costes operacionales del sistema.



DDD ayuda a definir límites del dominio mediante contextos delimitados, pero estos no equivalen necesariamente a unidades de despliegue independientes.

Figura 15: Relación entre desacoplamiento lógico y atributos de calidad. La arquitectura condiciona las decisiones posibles, pero las propiedades de despliegue y escalabilidad también dependen de la estructura de ejecución y de la infraestructura.

Conclusión

Para un sistema que exige elevada agilidad y escalabilidad, las arquitecturas Limpia y Hexagonal pueden constituir una base más adecuada cuando la prioridad es aislar las reglas del negocio, controlar las dependencias y facilitar la sustitución de detalles tecnológicos. Sin embargo, su elección no garantiza por sí sola un mejor rendimiento ni despliegues independientes. Estas propiedades deben conseguirse mediante decisiones de diseño, modularidad y operación coherentes con los requisitos de calidad.

La optimización de una Arquitectura N-Capas, por su parte, puede ser suficiente cuando el principal problema se concentra en operaciones concretas. Si el problema es estructural, las rutas rápidas y las capas abiertas no sustituyen la necesidad de revisar los límites entre componentes y la dirección de sus dependencias.

Referencias Bibliográficas

Cockburn, Alistair (2005). *Hexagonal Architecture (Ports and Adapters)*. URL: `https://alistair.cockburn.us/hexagonal-architecture/`.

De la Torre Llorente, César et al. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0*. Microsoft Ibérica.

Evans, Eric (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

Martin, Robert C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Palermo, Jeffrey (2008). *The Onion Architecture*. URL: `https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/`.

Richards, Mark (2015). *Software Architecture Patterns*. O'Reilly Media.

Richards, Mark y Neal Ford (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de uso de inteligencia artificial

Durante la elaboración del presente seminario se utilizaron herramientas de inteligencia artificial generativa como apoyo complementario para la comprensión de conceptos de arquitectura de software, la elaboración de fragmentos de código y la preparación del documento técnico. Su utilización se distribuyó de acuerdo con las siguientes finalidades:

- **DeepSeek (6.2.3) y Gemini (flash 3):** se utilizaron como herramientas de apoyo para aclarar conceptos clave relacionados con los estilos arquitectónicos estudiados, sus principios de diseño y sus implicaciones prácticas. Asimismo, se emplearon para apoyar la elaboración de bloques de código y ejemplos técnicos utilizados durante el desarrollo de las respuestas.
- **ChatGPT (modelo GPT-6):** se empleó como apoyo para la revisión y modificación del código fuente en LaTeX, la organización y reformulación de determinados fragmentos del documento y la elaboración del código de los diagramas arquitectónicos mediante TikZ. Estos diagramas se diseñaron para representar las relaciones entre componentes, las direcciones de las dependencias y las consecuencias de las decisiones arquitectónicas analizadas.

Integración de los resultados y responsabilidad académica

Los resultados generados mediante estas herramientas se utilizaron como material de apoyo para la elaboración del documento y se incorporaron de acuerdo con las necesidades del seminario. Su utilización no sustituye la consulta de las fuentes bibliográficas ni la fundamentación teórica de las decisiones arquitectónicas presentadas.

El equipo asume la responsabilidad final sobre el contenido del trabajo y debe verificar la exactitud técnica de las explicaciones, la pertinencia de los ejemplos y fragmentos de código, la correspondencia entre las citas y las fuentes consultadas, y el correcto funcionamiento del documento LaTeX. Asimismo, corresponde a sus integrantes comprobar que los diagramas y las conclusiones representen fielmente los conceptos estudiados y los requisitos establecidos en el enunciado del seminario.